

MINI NLP PROJECT REPORT

Submitted By

Name: ABHINAND AJIKUMAR

Register Number: 24UBC101

Class: III BCA A

Course: Natural Language Processing

1. Title

NLP-Based Movie Review Sentiment Analysis Using TF-IDF and Linear SVM

2. Problem Statement

Movie review websites contain a large amount of textual information expressing people's opinions about movies. These reviews may contain positive or negative opinions, informal expressions, long sentences, and different writing styles.

Manually analyzing thousands of movie reviews to determine whether they express a positive or negative opinion is time-consuming. Therefore, an automated system is useful for identifying the sentiment of movie reviews.

The objective of this project is to develop an NLP-based sentiment classification system that processes movie review text and automatically predicts whether the review is **Positive** or **Negative**.

The project uses **text preprocessing**, **TF-IDF feature extraction**, and a **Linear Support Vector Machine (SVM)** classifier to perform sentiment classification.

Problem Relevance

Movie review sentiment analysis is an important application of Natural Language Processing because it allows computers to analyze opinions expressed in textual data.

The system can be useful in:

- Movie review websites
- Recommendation systems

- Social media opinion analysis
- Customer feedback analysis
- Entertainment platforms
- Opinion mining systems

3. Objectives

The main objectives of this project are:

1. To develop an NLP-based movie review sentiment classification system.
2. To load and analyze the IMDb movie review dataset.
3. To preprocess and clean the review text.
4. To divide the dataset into training and testing portions.
5. To convert textual reviews into numerical features using TF-IDF.
6. To use both unigrams and bigrams for feature extraction.
7. To train a Linear Support Vector Machine classifier.
8. To classify movie reviews as Positive or Negative.
9. To evaluate the model using accuracy, precision, recall, F1-score, and confusion matrix.
10. To test the trained system on new and unseen movie reviews.

4. Dataset

4.1 Dataset Name

The project uses an **IMDb Movie Review Dataset** containing movie reviews labelled according to their sentiment.

4.2 Dataset Description

The dataset contains textual movie reviews along with their corresponding sentiment labels.

Each review belongs to one of two sentiment categories:

- **Positive**
- **Negative**

The dataset contains a total of **50,000 movie reviews**.

4.3 Dataset Size

The dataset contains:

50,000 records and 2 columns

The two columns are:

- `review`
- `sentiment`

4.4 Dataset Attributes

Attribute	Description
review	Contains the textual movie review
sentiment	Indicates whether the review is positive or negative

Target Variable

The target variable is:

sentiment

It contains two classes:

- **Negative** – indicates an unfavourable opinion about the movie.
- **Positive** – indicates a favourable opinion about the movie.

4.5 Dataset Considerations

Movie reviews can contain **negation, sarcasm, strong opinions, informal expressions, and long sentences**. These characteristics make sentiment classification a useful NLP problem.

For example, words such as *excellent*, *wonderful*, and *enjoyed* may indicate positive sentiment, while words such as *boring*, *disappointing*, and *waste of time* may indicate negative sentiment.

5. Methodology

The overall methodology followed in this project is:

IMDb Dataset → Data Exploration → Text Preprocessing → Train-Test Split → TF-IDF Feature Extraction → Linear SVM Training → Prediction → Evaluation → Testing with New Reviews

5.1 Data Loading

The IMDb dataset was loaded into a Pandas DataFrame.

The dataset contains:

- **50,000 reviews**
- **2 columns**
- `review`
- `sentiment`

The first few records were displayed to verify that the dataset was loaded correctly.

The dataset shape obtained was:

(50000, 2)

5.2 Data Exploration

The dataset was examined to understand its structure and contents.

The following were checked:

- Dataset shape
- Column names
- First few records
- Sentiment distribution
- Cleaned review text

The dataset contains approximately equal numbers of positive and negative reviews.

The sentiment distribution showed:

- **20,000 negative reviews in the training portion**
- **20,000 positive reviews in the training portion**

This indicates that the training portion used for the initial distribution analysis was balanced between the two sentiment classes.

Dataset considerations

Movie reviews can contain negation, sarcasm, strong opinions, informal expressions, and long sentences. These characteristics make sentiment classification a useful NLP problem.

```
22] 2s ▶ # Load the uploaded IMDB dataset
dataset = pd.read_csv("/content/IMDB Dataset.csv")

print("Dataset loaded successfully!")

print("\nDataset shape:", dataset.shape)

print("\nDataset columns:", list(dataset.columns))

print("\nFirst 5 rows:")
display(dataset.head())
```

```
... Dataset loaded successfully!

Dataset shape: (50000, 2)

Dataset columns: ['review', 'sentiment']

First 5 rows:
```

	review	sentiment
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend tl...	positive
3	Basically there's a family where a little boy ...	negative
4	Petter Matter's "Love in the Time of Money" is...	positive

6. Text Preprocessing

Before applying machine learning, the movie reviews were cleaned and prepared for feature extraction.

The preprocessing produced a new column called:

cleaned_text

The examples shown in the implementation demonstrate that unnecessary elements from the original reviews were removed.

For example, HTML elements such as:

```
<br />
```

were removed from the text.

The cleaned reviews were then used as input for the machine learning model.

Importance of Text Preprocessing

Raw movie reviews may contain unnecessary formatting, HTML tags, punctuation, and other elements that can create unwanted features.

Cleaning the text helps the model focus on useful textual patterns.

The implementation was checked to ensure that the cleaning process did not produce empty reviews.

The result was:

Empty cleaned reviews: 0

This means all processed reviews contained usable text after cleaning.

```
print("Empty cleaned reviews:", (train_df["cleaned_text"].str.strip() == "").sum())
display(train_df[["review", "cleaned_text", "sentiment"]].head(8))
```

Empty cleaned reviews: 0

	review	cleaned_text	sentiment
47808	I caught this little gem totally by accident b...	caught little gem totally accident back reviva...	positive
20154	I can't believe that I let myself into this mo...	believe let movie accomplish favor friends ask...	negative
43069	*spoiler alert! it just gets to me the nerve ...	spoiler alert gets nerve people remake use ter...	negative
19413	If there's one thing I've learnt from watching...	one thing learnt watching george romero creeps...	negative
13673	I remember when this was in theaters, reviews ...	remember theaters reviews said horrible well t...	negative
40368	Opera (the U.S. title is terror at the opera) ...	opera u title terror opera somewhat letdown da...	positive
15432	Heard about this film a long while ago and fin...	heard film long ago finally found ebay five bu...	positive
10208	John Holmes is so famous, he's infamous (as th...	john holmes famous infamous three amigos would...	positive

7. Train-Test Split

The original IMDb dataset contains **50,000 reviews**.

An initial split was performed to separate the official training and testing portions:

- **Training portion: 40,000 reviews**
- **Official test portion: 10,000 reviews**

The training portion was then further divided for model development.

An **80:20 train-test split** was applied to the 40,000-review training portion.

The split produced:

- **32,000 training samples**
- **8,000 testing samples**

The following settings were used:

- **Test size = 20%**
- **Random state = 42**
- **Stratification = y**

Stratification helps maintain a similar proportion of positive and negative reviews in both training and testing subsets.

The fixed random state also makes the experiment reproducible.

8. TF-IDF Feature Extraction

Machine learning algorithms cannot directly process raw text. Therefore, the cleaned movie reviews were converted into numerical feature vectors using **TF-IDF (Term Frequency–Inverse Document Frequency)**.

TF-IDF represents the importance of words in a document relative to the complete collection of documents.

It considers two main concepts:

Term Frequency (TF)

TF represents how frequently a particular word appears in a review.

Inverse Document Frequency (IDF)

IDF gives more importance to words that are relatively uncommon across the collection of documents.

The project used the following TF-IDF configuration:

- `stop_words = "english"`
- `ngram_range = (1, 2)`
- `min_df = 2`
- `max_df = 0.95`
- `sublinear_tf = True`
- `max_features = 100000`

Unigrams and Bigrams

Both **unigrams** and **bigrams** were used.

A **unigram** represents an individual word.

Example:

`excellent`

A **bigram** represents a two-word combination.

Example:

`very good`

Using both allows the model to capture individual words as well as short word combinations that may provide useful sentiment information.

Feature Extraction Result

The TF-IDF vectorizer generated a maximum of:

100,000 features

The resulting feature matrices were:

Dataset	Shape
Training TF-IDF	(32000, 100000)
Testing TF-IDF	(8000, 100000)

The vectorizer was fitted only on the training data and then applied to the testing data. This helps prevent information leakage from the test set into the training process.

8. Train-Test Split

An 80:20 stratified split is applied to the official training portion. Stratification keeps the proportion of positive and negative reviews approximately equal in both subsets.

A fixed random state is used so that the experiment can be reproduced.

```
1 X = train_df["cleaned_text"]
2 y = train_df["sentiment"]
3
4 X_train, X_test, y_train, y_test = train_test_split(
5     X,
6     y,
7     test_size=0.20,
8     random_state=42,
9     stratify=y
10 )
11
12 print("Training samples:", len(X_train))
13 print("Testing samples:", len(X_test))
```

9. Extracted Features

The extracted TF-IDF features represent words and word combinations found in the movie reviews.

The implementation generated:

100,000 extracted features

Examples of extracted features included terms such as:

- aaron
- aaron altman
- aaron carter
- abandoned
- abandoned building
- abandoned film
- abandoned hotel
- abandoned house
- abandoned school

This demonstrates that the feature extraction process captured both individual words and two-word combinations.

9. TF-IDF Feature Extraction

TF-IDF represents each review as a numerical vector based on the importance of its words.

Both **unigrams** and **bigrams** are used. Unigrams represent individual words, while bigrams capture short two-word expressions. This can help preserve useful phrases in sentiment classification.

The vectorizer is fitted only on the training data and then applied to the test data to prevent information leakage.

```
tfidf_vectorizer = TfidfVectorizer(  
    stop_words="english",  
    ngram_range=(1, 2),  
    min_df=2,  
    max_df=0.95,  
    sublinear_tf=True,  
    max_features=100000  
)  
  
X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)  
  
X_test_tfidf = tfidf_vectorizer.transform(X_test)  
  
print("Training TF-IDF shape:", X_train_tfidf.shape)  
print("Testing TF-IDF shape:", X_test_tfidf.shape)  
  
... Training TF-IDF shape: (32000, 100000)  
Testing TF-IDF shape: (8000, 100000)
```

```
feature_names = tfidf_vectorizer.get_feature_names_out()  
  
print("Number of extracted features:", len(feature_names))  
  
print("\nFirst 40 features:")  
  
print(feature_names[:40])  
  
Number of extracted features: 100000  
  
First 40 features:  
['aa' 'aaa' 'aag' 'aaliyah' 'aames' 'aamir' 'aamir khan' 'aapke'  
 'aapke hain' 'aardman' 'aaron' 'aaron altman' 'aaron carter'  
 'aaron eckhart' 'aaron sorkin' 'ab' 'aback' 'abandon' 'abandoned'  
 'abandoned building' 'abandoned film' 'abandoned hotel' 'abandoned house'  
 'abandoned school' 'abandoning' 'abandonment' 'abandons' 'abba' 'abbas'  
 'abbey' 'abbie' 'abbot' 'abbot costello' 'abbot white' 'abbott'  
 'abbott costello' 'abbreviated' 'abby' 'abc' 'abc family']
```

10. Linear SVM Classification

A **Linear Support Vector Machine (Linear SVM)** was used for sentiment classification.

Linear SVM is well suited for text classification because text represented using TF-IDF generally produces a **high-dimensional and sparse feature matrix**.

The classifier learns a decision boundary that separates positive and negative movie reviews based on their extracted textual features.

The model was created using:

LinearSVC(random_state=42)

The model was trained using:

- TF-IDF training features
- Training sentiment labels

After training, the model successfully generated predictions for the testing data.

11. Model Evaluation

The trained Linear SVM model was evaluated using several classification metrics.

The following evaluation measures were used:

1. Accuracy
2. Precision
3. Recall
4. F1-score
5. Classification report
6. Confusion matrix

These metrics provide a more complete understanding of how well the classifier distinguishes between positive and negative reviews.

10. Linear SVM Classification

A Linear Support Vector Machine (Linear SVM) is used for sentiment classification. Linear SVM is well suited to high-dimensional sparse text representations such as TF-IDF.

The classifier learns a decision boundary that separates positive and negative reviews using the extracted textual features.

```
from sklearn.svm import LinearSVC

model = LinearSVC(random_state=42)

model.fit(X_train_tfidf, y_train)

print("Linear SVM training completed successfully!")

Linear SVM training completed successfully!

y_pred = model.predict(X_test_tfidf)

comparison = pd.DataFrame({
    "Actual": y_test.map(sentiment_names).values,
    "Predicted": pd.Series(y_pred).map(sentiment_names).values
})

print("Predictions generated:", len(y_pred))

display(comparison.head(20))
```

... Predictions generated: 8000

	Actual	Predicted
0	NaN	NaN
1	NaN	NaN
2	NaN	NaN
3	NaN	NaN

11.1 Accuracy

Accuracy represents the proportion of correctly classified reviews among all the testing reviews.

The model achieved:

Accuracy = 89.95%

This means that approximately 90 out of every 100 test reviews were classified correctly.

11.2 Precision

Precision measures how many reviews predicted as a particular class actually belonged to that class.

The obtained macro precision was:

Precision = 89.98%

A high precision indicates that the model generally makes reliable sentiment predictions.

11.3 Recall

Recall measures how many of the actual reviews belonging to a particular sentiment class were correctly identified.

The obtained macro recall was:

Recall = 89.95%

A high recall indicates that the model successfully identifies most of the positive and negative reviews.

11.4 F1-Score

F1-score is the harmonic mean of precision and recall.

It provides a balanced measure of classification performance.

The obtained F1-score was approximately:

F1-Score = 89.95%

The similar values of precision, recall, and F1-score indicate that the model performs reasonably consistently across the two sentiment classes.

11.3 Score: Balance between precision and recall

- **Classification report:** Provides class-wise performance.
- **Confusion matrix:** Shows correct and incorrect predictions for both classes.

Macro averaging is used for the main summary so that both sentiment classes receive equal importance.

```
accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(
    y_test, y_pred,
    average="macro",
    zero_division=0
)

recall = recall_score(
    y_test, y_pred,
    average="macro",
    zero_division=0
)

f1 = f1_score(
    y_test, y_pred,
    average="macro",
    zero_division=0
)

print("Accuracy :", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall   :", round(recall, 4))
print("F1-Score :", round(f1, 4))

Accuracy : 0.8995
Precision: 0.8998
Recall   : 0.8995
F1-Score : 0.8995
```

12. Classification Report

The classification report provides detailed performance for both sentiment classes.

Class	Precision	Recall	F1-Score	Support
Negative	0.91	0.88	0.90	4000
Positive	0.89	0.91	0.90	4000
Accuracy			0.90	8000
Macro Avg	0.90	0.90	0.90	8000
Weighted Avg	0.90	0.90	0.90	8000

The results show that the model performs well for both classes.

The model has slightly higher **recall for positive reviews (0.91)**, while the **negative class has slightly higher precision (0.91)**.

```
print(classification_report(  
    y_test,  
    y_pred,  
    target_names=["negative", "positive"],  
    zero_division=0  
))
```

	precision	recall	f1-score	support
negative	0.91	0.88	0.90	4000
positive	0.89	0.91	0.90	4000
accuracy			0.90	8000
macro avg	0.90	0.90	0.90	8000
weighted avg	0.90	0.90	0.90	8000

13. Confusion Matrix

A confusion matrix was used to understand the correct and incorrect predictions made by the Linear SVM classifier.

The confusion matrix obtained was:

Actual / Predicted	Negative	Positive
Negative	3539	461
Positive	343	3657

Interpretation

- **3539** negative reviews were correctly classified as negative.
- **461** negative reviews were incorrectly classified as positive.
- **343** positive reviews were incorrectly classified as negative.
- **3657** positive reviews were correctly classified as positive.

Therefore, the model correctly classified:

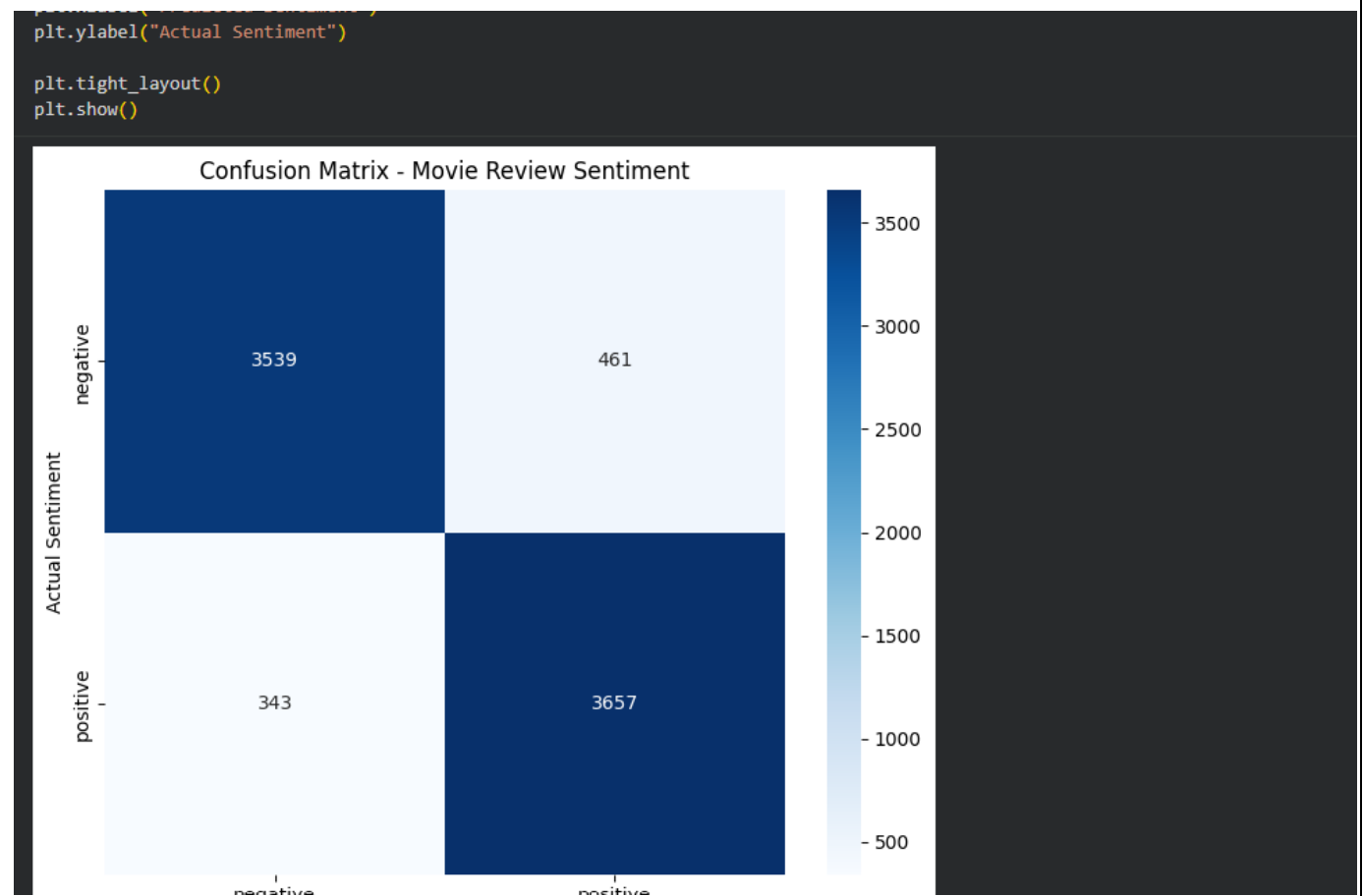
3539 + 3657 = 7196 reviews

out of **8000 testing reviews**.

The diagonal values represent correct classifications, while the off-diagonal values represent incorrect classifications.

The number of errors was:

461 + 343 = 804 reviews



14. Testing the System on New Reviews

After training and evaluating the classifier, the system was tested using new movie reviews that were not part of the training or testing dataset.

The prediction function applies the same text cleaning process and then transforms the review using the previously trained TF-IDF vectorizer before passing it to the Linear SVM model.

Test Review 1

Review:

"The movie was fantastic, emotional and beautifully acted."

Predicted Sentiment: Positive

Test Review 2

Review:

"I disliked this movie because the story was boring and extremely slow."

Predicted Sentiment: Negative

Test Review 3

Review:

"The performances were excellent and I really enjoyed every scene."

Predicted Sentiment: Positive

Test Review 4

Review:

"The film was disappointing and felt like a waste of time."

Predicted Sentiment: Negative

13. Test the System on New Reviews

The trained classifier can be used to predict sentiment for reviews that were not included in the training or testing data. The prediction function applies the same preprocessing and TF-IDF transformation before classification.

```
def predict_sentiment(review):
    cleaned = clean_text(review)
    vectorized = tfidf_vectorizer.transform([cleaned])
    prediction = model.predict(vectorized)[0]

    return prediction

new_reviews = [
    "The movie was fantastic, emotional and beautifully acted.",
    "I disliked this movie because the story was boring and extremely slow.",
    "The performances were excellent and I really enjoyed every scene.",
    "The film was disappointing and felt like a waste of time."
]

for review in new_reviews:
    print("Review:", review)
    print("Predicted Sentiment:", predict_sentiment(review))
    print("-" * 75)
```

```
Review: The movie was fantastic, emotional and beautifully acted.
Predicted Sentiment: positive
-----
Review: I disliked this movie because the story was boring and extremely slow.
Predicted Sentiment: negative
-----
Review: The performances were excellent and I really enjoyed every scene.
Predicted Sentiment: positive
-----
Review: The film was disappointing and felt like a waste of time.
Predicted Sentiment: negative
```

User Input Testing

The system was also designed to accept a movie review directly from the user.

For example:

Input:

"it was an average film"

Predicted Sentiment:

Negative

This demonstrates that the trained model can be used interactively to classify new movie reviews.

```
user_review = input("Enter a movie review: ")

result = predict_sentiment(user_review)

print("\nPredicted Sentiment:", result)

Enter a movie review: it was an average film

Predicted Sentiment: negative
```

15. Results Summary

15. Results and Analysis

The project provides an end-to-end sentiment analysis pipeline. Movie reviews are cleaned and transformed into TF-IDF features containing unigram and bigram information. A Linear SVM classifier is then trained to distinguish between positive and negative sentiment.

The executed evaluation cells provide the actual accuracy, precision, recall, and F1-score. The classification report gives class-level results, while the confusion matrix helps identify the types of sentiment errors made by the classifier.

The new-review testing section demonstrates how the trained NLP system can be used on unseen text.

```
[47]
✓ Os
▶ print("Final Model Performance")

print("=" * 35)

print(f"Accuracy : {accuracy:.4f}")
print(f"Precision : {precision:.4f}")
print(f"Recall : {recall:.4f}")
print(f"F1-Score : {f1:.4f}")
```

```
... Final Model Performance
=====
Accuracy : 0.8995
Precision : 0.8998
Recall : 0.8995
F1-Score : 0.8995
```

16. Strengths of the System

The developed system has several advantages:

16.1 Good Classification Performance

The model achieved approximately **89.95% accuracy**, showing that TF-IDF combined with Linear SVM is effective for movie review sentiment classification.

16.2 Suitable for Text Data

Linear SVM works well with high-dimensional sparse features such as TF-IDF representations.

16.3 Uses Bigrams

Using both unigrams and bigrams allows the system to capture short phrases in addition to individual words.

16.4 Balanced Dataset Split

Stratified splitting helps maintain a similar distribution of positive and negative reviews in the training and testing data.

16.5 Simple and Efficient

Compared with more complex deep learning models, Linear SVM can provide strong performance while remaining relatively straightforward to implement.

17. Limitations

Although the developed system provides good sentiment classification performance, it has some limitations.

17.1 Difficulty Understanding Context

The model mainly learns statistical relationships between words and sentiment. It does not deeply understand the meaning or context of a complete review.

17.2 Sarcasm

Sarcastic reviews can be difficult to classify correctly because the literal words may suggest a different sentiment from the writer's actual opinion.

17.3 Negation

Sentences containing complex negation may sometimes be challenging.

For example:

"The movie was not very good."

The presence of the word *good* alone may not completely represent the negative meaning of the sentence.

17.4 Unseen Vocabulary

If a review contains words or expressions that were not sufficiently represented in the training data, the model may have difficulty making an accurate prediction.

17.5 Limited Feature Representation

TF-IDF represents words statistically and does not capture the deeper semantic relationships that modern language models can understand.

18. Future Scope

The project can be improved in several ways.

18.1 Comparison with Other Algorithms

The Linear SVM model can be compared with:

- Logistic Regression
- Multinomial Naive Bayes
- Random Forest
- Decision Tree
- K-Nearest Neighbours

This would help identify the most suitable algorithm for movie sentiment classification.

18.2 Advanced NLP Techniques

Additional preprocessing methods such as:

- Stemming
- Lemmatization
- Improved tokenization
- Handling negation

could be explored.

18.3 Deep Learning

More advanced models such as:

- LSTM
- GRU

- CNN
- Transformer-based models

could be used to capture more complex language patterns.

18.4 Real-Time Application

The trained model could be integrated into an application where users enter a movie review and receive its predicted sentiment immediately.

19. Conclusion

This project developed an **NLP-based Movie Review Sentiment Analysis system** using text preprocessing, TF-IDF feature extraction, and a Linear Support Vector Machine classifier.

The IMDb dataset contained **50,000 movie reviews**, which were explored and prepared before model training. The reviews were cleaned and divided into training and testing data. The training portion was further split into **32,000 training samples and 8,000 testing samples** using stratified splitting.

The cleaned reviews were converted into numerical representations using **TF-IDF**, with both unigrams and bigrams used as features. The TF-IDF vectorizer generated up to **100,000 features**.

A **Linear SVM classifier** was then trained using these features. The model achieved an accuracy of **89.95%** on the testing data, with precision, recall, and F1-score also close to 90%.

The confusion matrix showed that the model correctly classified **7,196 out of 8,000** test reviews. The system was also tested with new movie reviews and successfully predicted positive and negative sentiments.

Overall, this project demonstrates how **Natural Language Processing and Machine Learning** can be combined to solve a practical text classification problem. It provided practical experience in dataset exploration, text preprocessing, TF-IDF feature extraction, machine learning model training, evaluation, confusion matrix analysis, and prediction of unseen reviews.

20. References

1. IMDb Movie Review Dataset used for the project.
2. Scikit-learn documentation – TF-IDF Vectorization.
3. Scikit-learn documentation – Linear Support Vector Classification.
4. Scikit-learn documentation – Classification Metrics.
5. Python documentation.

